AI Resume Screening System

Deployed on AWS using EC2, S3, DynamoDB and IAM

Python | Streamlit | TF-IDF | Amazon Web Services

Team members

Parth Wankhede (Roll No. 58)

Shashwat Deshpande (Roll No. 64)

Soumya Dhole (Roll No. 65)

Trigun Jagal (Roll No. 66)

Problem Statement and Objectives

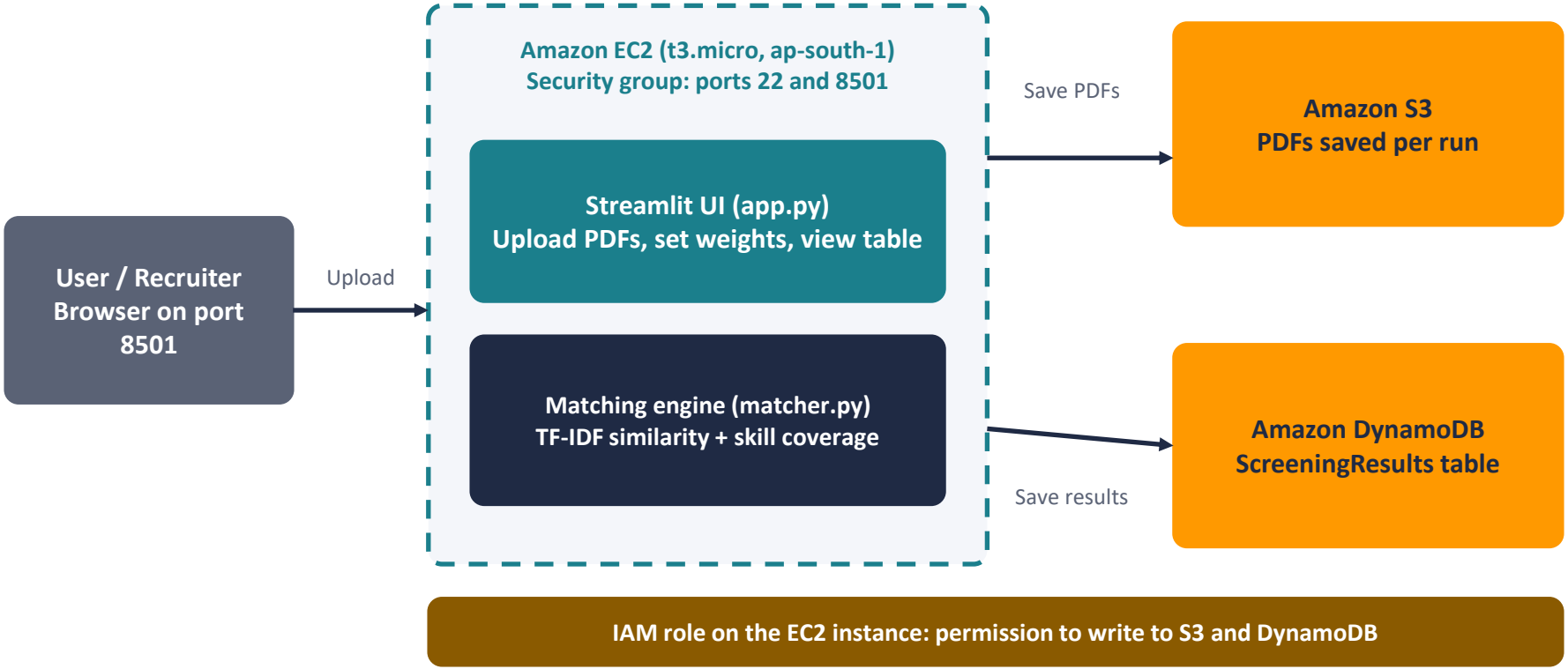
Problem

Recruiters receive many resumes for one job. Reading each one manually is slow, inconsistent and hard to repeat. Candidates also get no clear view of which required skills are missing from their resume.

Objectives

- Rank PDF resumes against a job description
- Show matched and missing skills for every resume
- Let the user tune the score weights
- Store uploaded PDFs and results on AWS
- Run the app on a cloud server anyone can open
- Keep screening fair: no sensitive attributes are scored

System Architecture



Technology Stack and Cloud Services

Application

Python, Streamlit web interface, scikit-learn TF-IDF vectorizer, PDF text extraction, pandas for the results table and CSV download.

Matching logic

Overall score = TF-IDF similarity (weight 60% by default) + skills coverage (40%). The weights are adjustable with a slider and always total 100%.

Amazon EC2

Hosts the Streamlit app on a t3.micro instance in Mumbai (ap-south-1).

Amazon S3

Stores each uploaded PDF under resumes/<run id>/.

Amazon DynamoDB

Stores scores, matched and missing skills per resume.

AWS IAM

Role attached to EC2 grants access to S3 and DynamoDB.

Implementation: How a Resume Is Scored

1

Read PDFs

Text is extracted from each uploaded resume

2

TF-IDF

Resume and job description become weighted term vectors; similarity is measured

3

Skills check

Resume text is compared with an editable skill dictionary; matched and missing skills are listed

4

Combine

Overall score = weighted mix of similarity and skills coverage

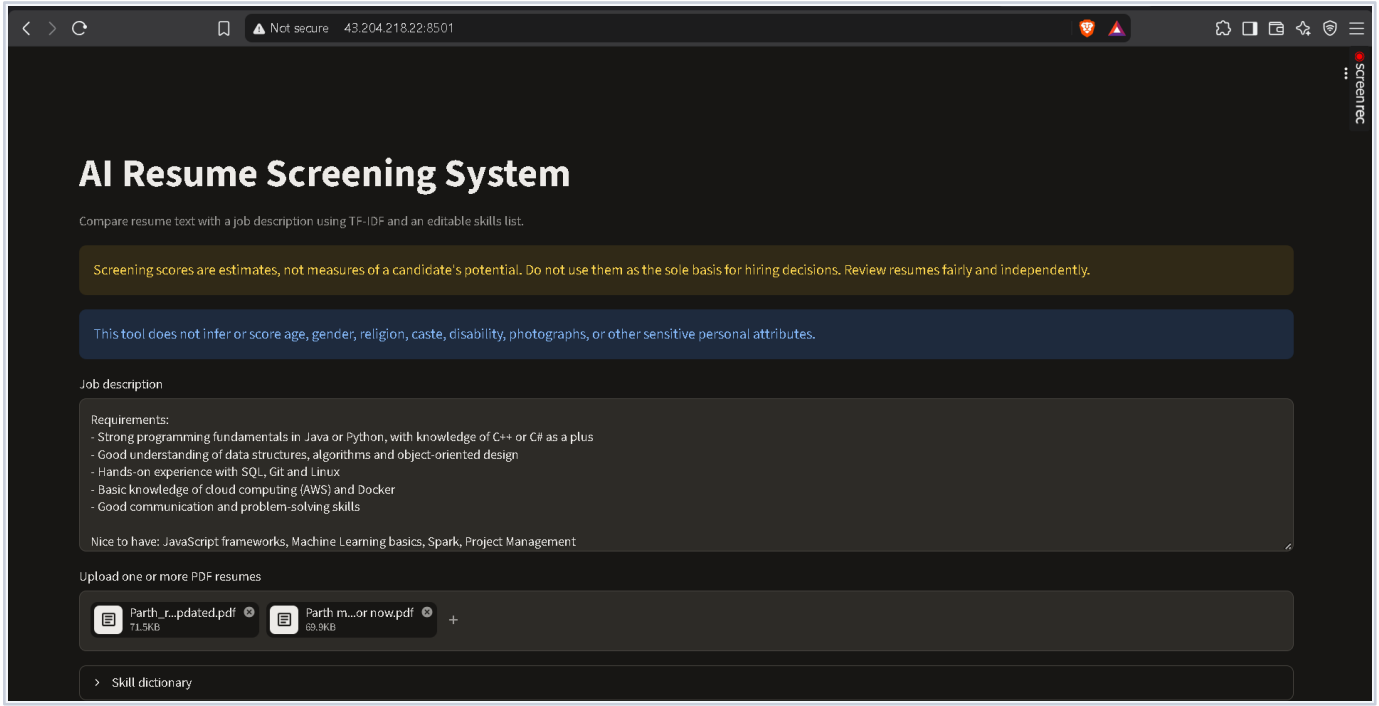
5

Save

PDFs go to S3, result rows go to DynamoDB, table is ranked on screen

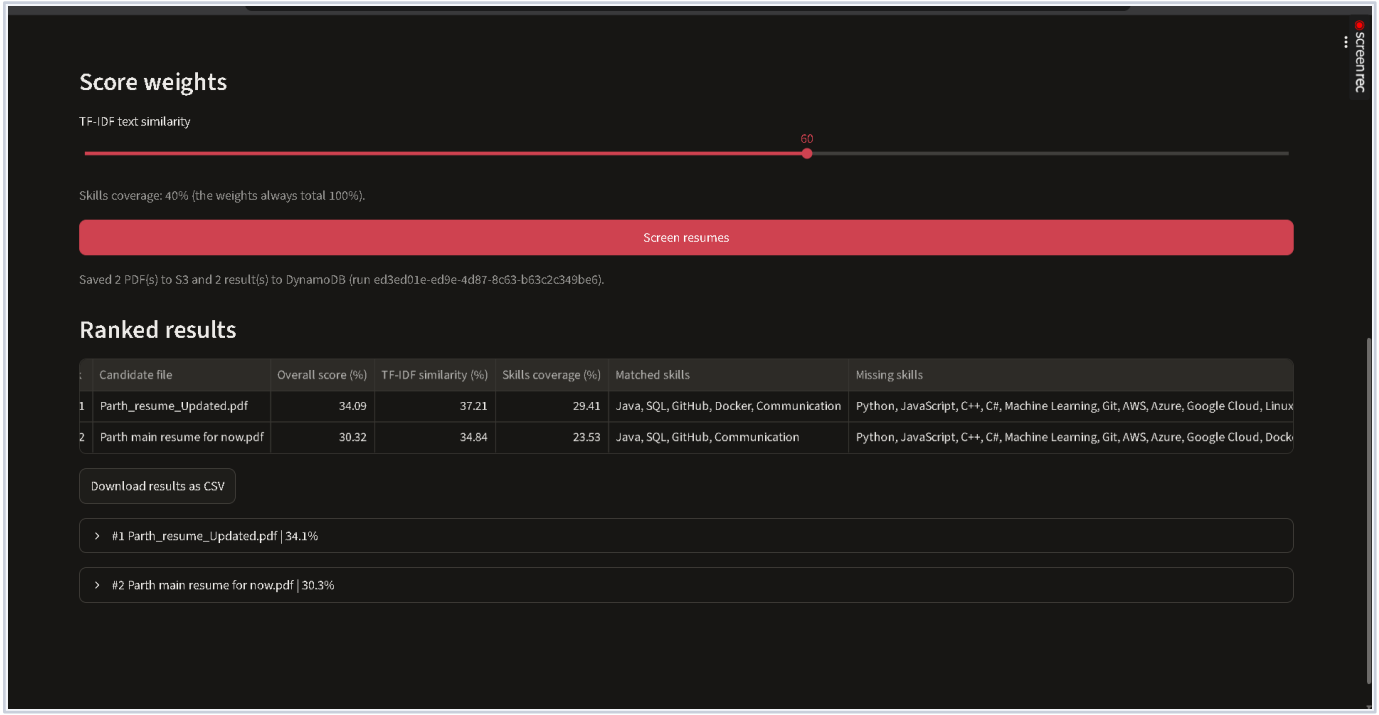
Worked example from the live run: 0.60 x 37.21 + 0.40 x 29.41 = 34.09 overall score for the top-ranked resume.

Application: Job Description and Resume Upload



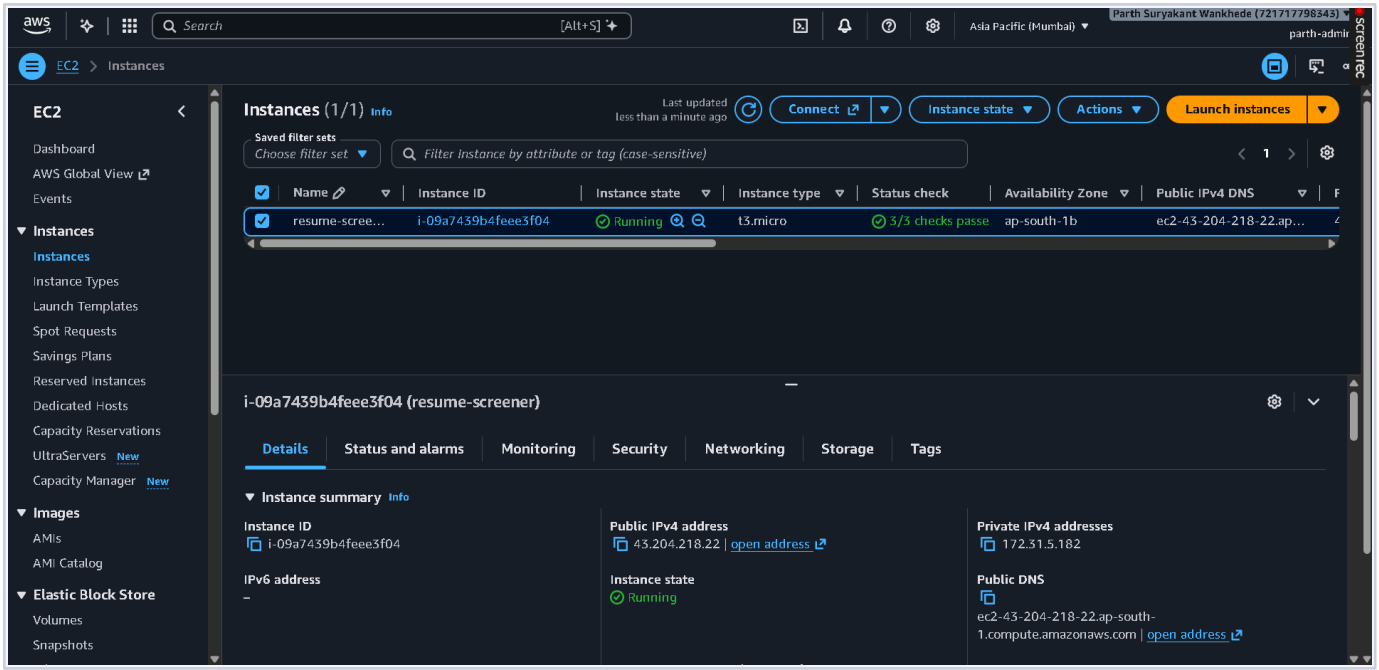
- Job description pasted into the text box
- Two PDF resumes uploaded
- Fairness notices shown at the top of the page
- Editable skill dictionary below the upload box
- Opened from the public EC2 address on port 8501

Application: Ranked Results



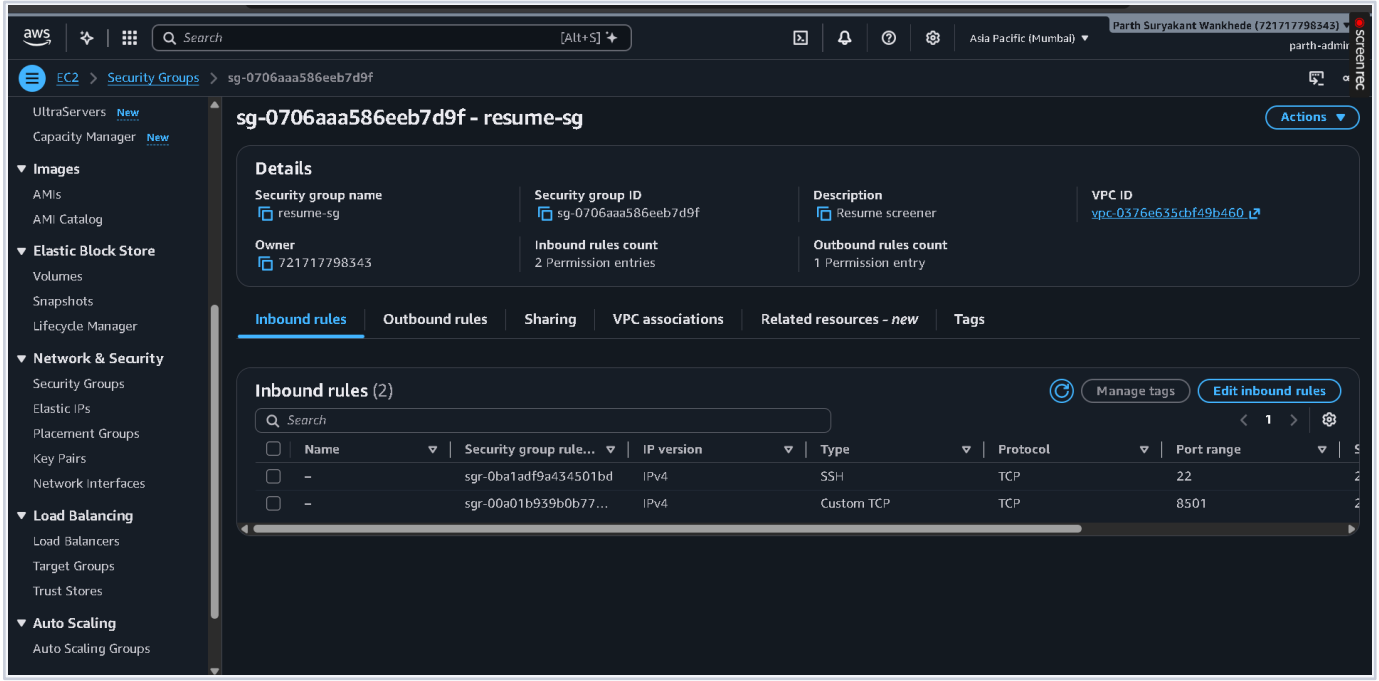
- Weight slider: 60% TF-IDF, 40% skills
- Message confirms the PDFs went to S3 and the results to DynamoDB
- Table ranks resumes by overall score
- Matched and missing skills listed per resume
- Results can be downloaded as CSV

Cloud Deployment: Amazon EC2



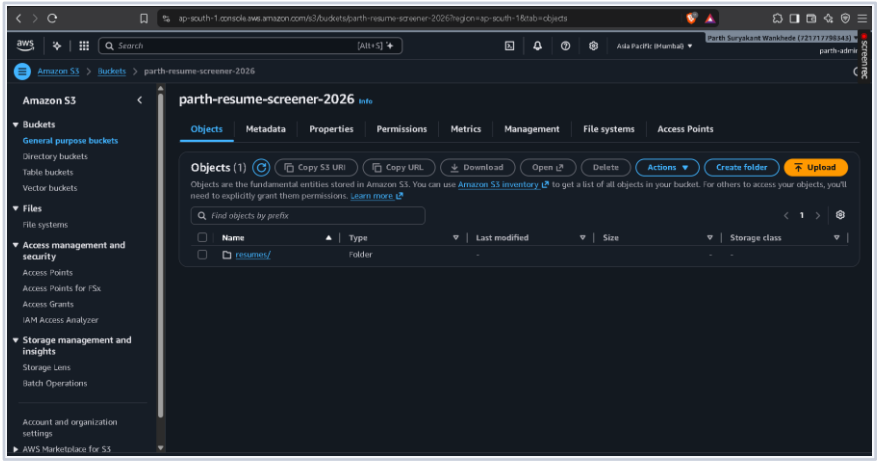
- Instance type: t3.micro
- State: Running, 3/3 status checks passed
- Availability zone: ap-south-1b (Mumbai)
- Public IPv4 address serves the app
- Streamlit runs on this instance

Cloud Deployment: Security Group

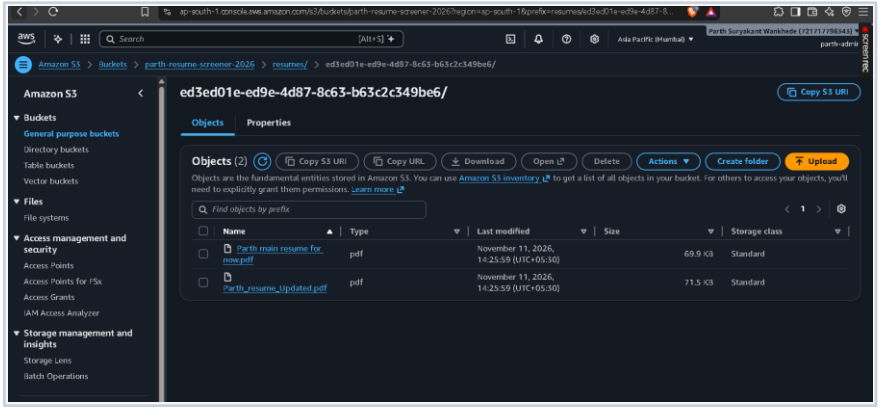


- Security group: resume-sg
- Inbound rule 1: SSH, TCP port 22, for administration
- Inbound rule 2: Custom TCP port 8501, for the Streamlit app
- All other inbound traffic is blocked

Cloud Storage: Amazon S3



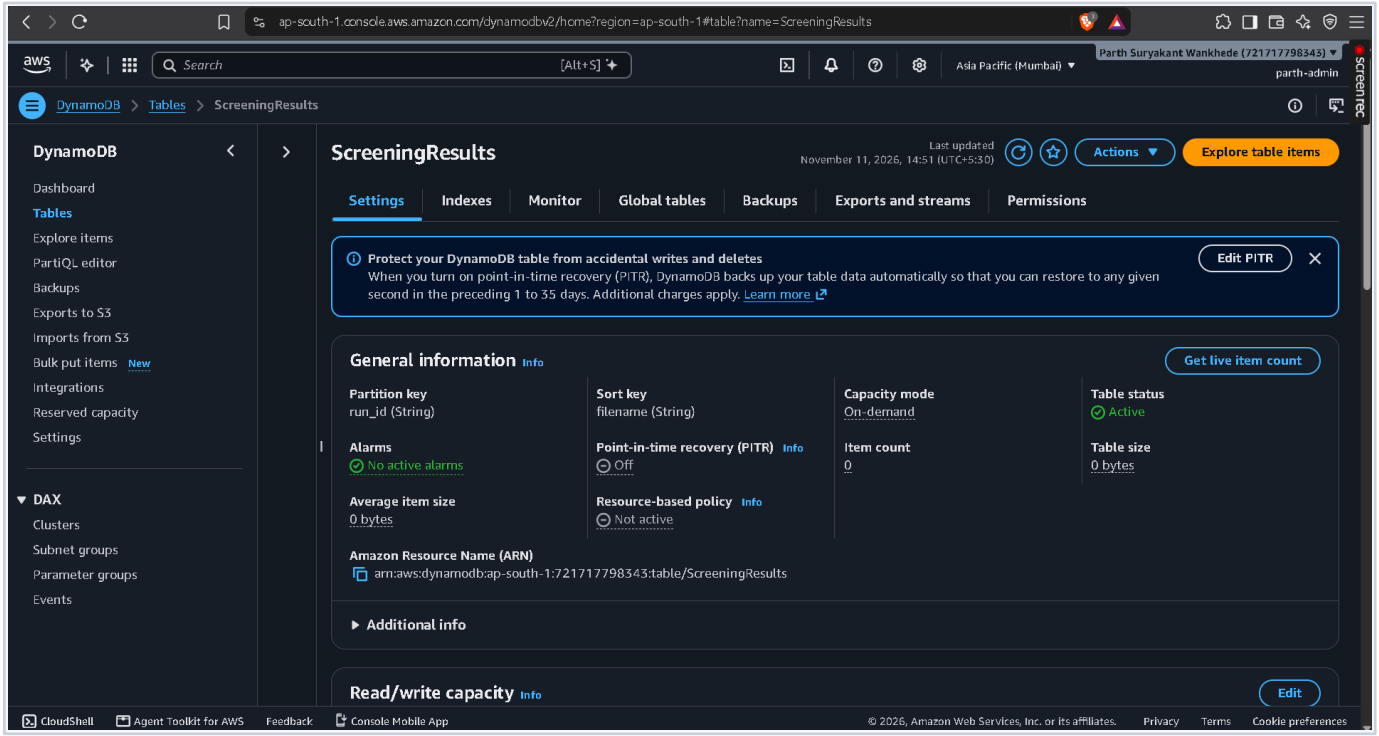
Bucket with the resumes/ folder



One folder per run, holding the uploaded PDFs

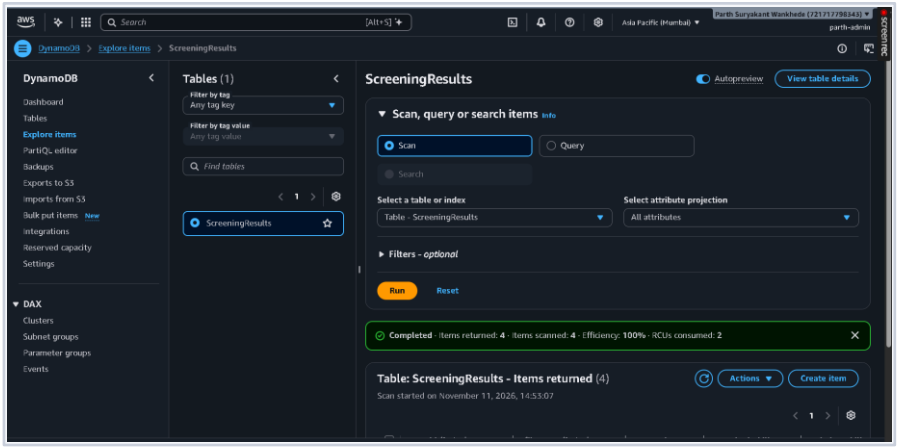
- Each screening run gets its own folder named with a run ID
- The same run ID appears in the app message and in DynamoDB, so PDFs and scores can be matched

Cloud Database: Amazon DynamoDB

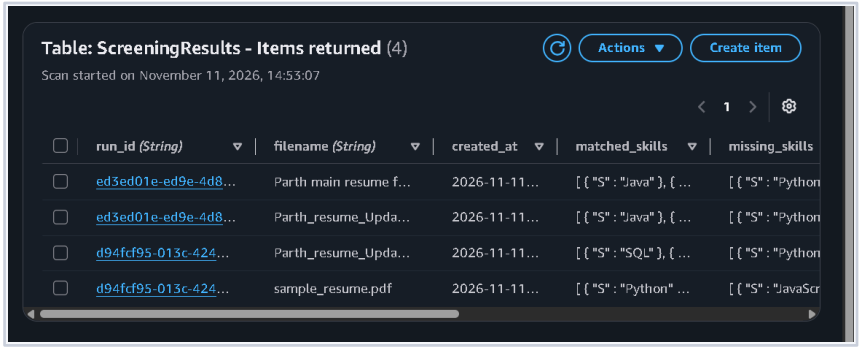


- Table: ScreeningResults
- Partition key: run_id
- Sort key: filename
- Capacity mode: On-demand
- Status: Active

Stored Results in DynamoDB



Scan of the table: 4 items returned



Items with run ID, filename, time and skills

- Each resume in a run is saved as one item
- Matched skills and missing skills are saved with the score

Deployment Steps

1. Create the S3 bucket and the DynamoDB table (ScreeningResults) in ap-south-1
2. Create an IAM role for EC2 with access to S3 and DynamoDB, and attach it to the instance
3. Launch a t3.micro EC2 instance and a security group opening ports 22 and 8501
4. Connect over SSH, clone the GitHub repository and install requirements
5. Start the Streamlit app on port 8501
6. Open the public address in a browser, upload PDFs and verify the data in S3 and DynamoDB

Results



- The app ranked both resumes and listed their matched and missing skills
- The PDFs and result rows were saved to S3 and DynamoDB for the run
- Scores are modest because the job description lists many skills (Python, AWS, Linux and others) that were not found in the resumes
- Scores are estimates and should support, not replace, human review

Challenges and Future Scope

Challenges

Setting up IAM permissions so the EC2 instance can reach S3 and DynamoDB without stored keys.

Opening only the needed ports in the security group.

Extracting text from PDFs: scanned (image-only) PDFs have no text and cannot be scored.

Future scope

Add AWS Lambda to validate uploads automatically.

Add CloudWatch logs and alarms for monitoring.

Use HTTPS with a domain name and a load balancer.

Support scanned PDFs with OCR and use semantic embeddings for matching.

Conclusion

- A working resume screening app is live on AWS EC2.
- It ranks PDF resumes with TF-IDF similarity and skill coverage.
- S3 keeps the uploaded PDFs and DynamoDB keeps the scores.
- IAM and a security group keep access controlled.

Thank you